Norma IEEE 754

Organización de Computadoras
Clase 22

Mg. Javier Echaiz
D.C.I.C. – U.N.S.
http://cs.uns.edu.ar/~jechaiz
je@cs.uns.edu.ar

Repaso

Repaso Exceso 3

X = 0298, Y = 0567
Realizar X – Y en S-M y en C10.

```
                  1   1
  567        1000 1001 1010
- 298        0101 1100 1011
  9269       0010 1100 1111
               +11  -11  -11
             1100 0101 1001 1100 = 9269SM
```

SGN de la mayor magnitud

```
                  1   1
  0298       0011 0101 1100 1011
+ 9433       1100 0111 0110 0110
  9731C10    1111 1101 0011 0001
  -269        -11  -11  +11  +11
             1100 1010 0110 0100 = 9731C10
```


Repaso Exceso 3

X = 0395, Y = 0105. Y' = 9894_{C9}
Realizar X – Y en C9.

```
  1                1   1   1
  0395       0011 0110 1100 1000
+ 9894       1100 1011 1100 0111
  0289       0000 0010 1000 1111
   +1         +11  +11  +11  -11
  0290C9      0011 0101 1011 1100
                                 +1
             0011 0101 1011 1101
              0    2    8    10
              0    2    9    0
```

¿Cómo puede evitarse este "desagradable error"?

Terminando 1ro el algoritmo de C9.

¿10?

"Padre" del estándar de PF

IEEE Standard 754

1989
ACM Turing
Award Winner!

Prof. W. Kahan

www.cs.berkeley.edu/~wkahan/
…/ieee754status/754story.html

Representación en IEEE-754

Simple Precisión - 32 bits

s	exponente	mantisa
1 bit	8 bits	23 bits

Doble Precisión - 64 bits

s	exponente	mantisa
1 bit	11 bits	52 bits

IEEE-754

0	00000000	00000000000000000000000
s g n	exponente	mantisa (*significand*)

$$(-1)^{S} * 1.M * 2^{E-127}$$

Signo

1 es asumido

Mantisa (sin *leading* 1)

Base

Exponente

¿Cómo representamos el 0?

0	00000000	00000000000000000000000
s g n	exponente	mantisa (*significand*)

$$(-1)^{S} * 1.M * 2^{E-127}$$

	E == 0	0 < E < 255	E == 255
M==0	0	Potencias de 2	±∞
M!=0	No Normalizados, típicamente underflow	Números convencionales	Not a Number

Representación de Denormals IEEE 754

- Expresión "usual" (*slide* anterior) de punto flotante:

  $(-1)^{S}$ x (1 + mantisa) x $2^{(Exp_{representado} - 127)}$

- If (fp.exp == 0 and fp.mantisa != 0)

  Denorm!!!

  $(-1)^{S}$ x (0 + mantisa) x $2^{(-126)}$

  Permite la degradación gradual hacia 0.

Exponente IEEE-754

- El exponente emplea 8 bits.
- *bias* = 127.
  - El exponente se trata como un binario no signado de 8 bits y se le resta 127 para conocer su valor real.
- Ejemplos:

  `00000001` representa –126.

  `10000000` representa +1.

  `11111110` representa +127.

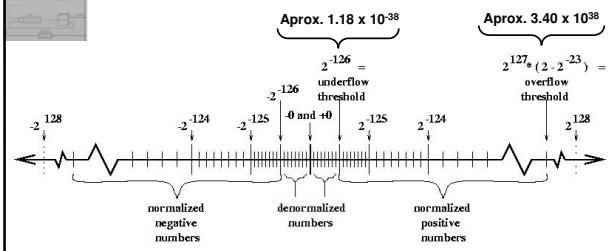
Análisis tabla anterior

Real Number and NaN Encodings For 32-Bit Floating-Point Format

S	E	F		S	E	F	
1	0	0	–0	0	0	0	+0
1	0	0.XXX^2	–Denormalized Finite	0	0	0.XXX^2	+Denormalized Finite
1	1...254	Any Value	–Normalized Finite	0	1...254	Any Value	+Normalized Finite
1	255	0	–∞	0	255	0	+∞
X[1]	255	1.0XX^2	–SNaN	X	255	1.0XX^2	+SNaN
X[1]	255	1.1XX	–QNaN	X	255	1.1XX	+QNaN

NOTES:
1. Sign bit ignored.
2. Fractions must be non-zero.

IEEE-754



Normalización en IEEE-754

Normalizado: un único *dígito* a la izquierda del punto. Debe ser un 1!

Le seguimos llamando *dígito* a pesar de ser un "0" o un "1".

101.0111×2^{13}

normalizado

1.010111×2^{15}

$1.010111 \times 2^{00001111}$

exponentes en binario!

Representación de Denormals IEEE 754

- Expresión "usual" (*slide* anterior) de punto flotante:
  $(-1)^S \times (1 + \text{mantisa}) \times 2^{(Exp_{representado} - 127)}$

- If (fp.exp == 0 and fp.mantisa != 0)
  Denorm!!!
  $(-1)^S \times (0 + \text{mantisa}) \times 2^{(-126)}$

Conversión a la Norma

- ***Ejemplo 1***: Convertir 10.666015625_{10} a la norma IEEE de punto flotante:

1. $10_{10} = 1010_2$
2. $0.666015625_{10} = .101010101_2$
3. $1010.101010101 \times 2^0$
4. $1.010101010101 \times 2^3$
5. Exponente 3 = 130 - 127 = 10000010
   Es decir, $Exp_{representado} = Exp_{real} + 127 = 10000010$
6. 0 10000010 01010101010100000000000
   signo, exponente, mantisa

Ejemplos Extremos

sgn	exponente	mantisa (*significand*)
0	00000000	00000000000000000000000

$(-1)^S * 1.M * 2^{E-127}$

Fin del Repaso

```
0 11111111 00000100000000000000000 = NaN
1 11111111 00100010001001010101010 = NaN

0 11111111 00000000000000000000000 = Infinity
0 10000001 10100000000000000000000 = +1 * 2^(129-127) * 1.101 = 6.5
0 10000000 00000000000000000000000 = +1 * 2^(128-127) * 1.0 = 2
0 00000001 00000000000000000000000 = +1 * 2^(1-127) * 1.0 = 2^(-126)
0 00000000 10000000000000000000000 = +1 * 2^(-126) * 0.1 = 2^(-127)
0 00000000 00000000000000000000001
      = +1 * 2^(-126) * 0.00000000000000000000001
      = 2^(-149)  (positivo más pequeño)
0 00000000 00000000000000000000000 = +0
1 00000000 00000000000000000000000 = -0
1 10000001 10100000000000000000000 = -1 * 2^(129-127) * 1.101 = -6.5
1 11111111 00000000000000000000000 = -Infinity
```

(+) más pequeño norm.

(+) más pequeño normalizado

```
0 00000001 00000000000000000000000 = +1 * 2^(1-127) * 1.0 = 2^(-126)   A
```

Próximo positivo más pequeño (es un denormal)

```
0 00000000 11111111111111111111111 =                                   B
                    = +1 * 2^(-126) * 0.11111111111111111111111
```

¿Por qué no exponente = -127?

"convencionales"

A
B

(+) Denorm.

```
0 00000001 00000000000000000000000 =
                    = +1 * 2^(1-127) * 1.0 = 1.0 x 2^(-126)
0 00000000 11111111111111111111111
                    = +1 * 2^(-126) * 0.11111111111111111111111
```

0

Si el exponente fuese -127

$2^{(-127)} * 0.11111111111111111111111$ ó
$2^{(-126)} * 0.01111111111111111111111$

E == 0

M!=0 denorm.

Más sobre positivos pequeños

```
0 00000000 00000000000000000000001
      = +1 * 2^(-126) * 0.00000000000000000000001
      = 2^(-149)  (positivo más pequeño (denormal))
```

¿Cuál es el segundo positivo más pequeño?

```
0 00000000 00000000000000000000001
      = +1 * 2^(-126) * 0.00000000000000000000010
      = 2^(-148)  (2do positivo más pequeño (denormal))
```

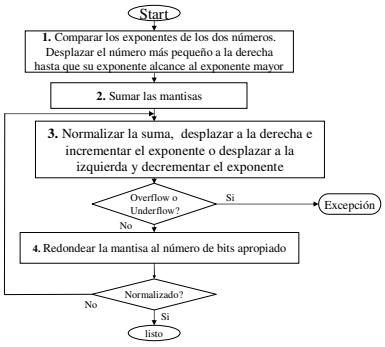
$-\infty$ ← 0 → $+\infty$

Suma en IEEE 754

Para sumar dos números en la norma de punto flotante se deben seguir los siguientes 4 pasos:

1. Alinear los puntos decimales ("desnormalizar" un número tal que ambos exponentes sean iguales) desplazar a la derecha la mantisa más pequeña por la diferencia de los exponentes.
2. Sumar las mantisas.
3. Renormalizar el resultado, chequeando un posible overflow en el exponente.
4. Redondear el resultado al número correcto de dígitos significativos (próxima clase).

Suma en IEEE 754



Suma en PF (std)

- ¿Cómo efectúa la suma en notación científica?
  $9.862 \times 10^4 + 5.492 \times 10^2$
- Algoritmo básico
  1. Alinear
  2. Sumar
  3. Normalizar
  4. Redondear (próxima clase)
- No exactamente…
  – Usualmente el redondeo "descarta" bits sobre el final.
  – Restar dos números casi iguales no provoca pérdida de bits, pero el resultado no tiene tantos bits significativos.

Multiplicación Punto Flotante

Se deben seguir los siguientes pasos para multiplicar dos números en punto flotante:

1. Sumar los exponentes (dado que están en exceso, se debe restar un exceso (127) para obtener un resultado *sesgado* (en exceso)).
2. Multiplicar las mantisas.
3. Normalizar el resultado y comprobar si hay overflow.
4. Redondear la respuesta (próxima clase).
5. Normalizar otra vez si es necesario.
6. Setear el signo.

Multiplicación en punto flotante

División en punto flotante

Precisión en IEEE-754

- La precisión es crucial en cálculos científicos.
- Los errores pequeños pueden acumularse…
- En IEEE-754 tenemos cuatro "modos de redondeo":
  - Hacia arriba (hacia +∞).
  - Hacia abajo (hacia -∞).
  - Hacia cero (truncado).
  - Proximidad (**Unbiased**).
    En caso de "empate" (mitad), redondear al próximo par.

Más frecuente

- Requiere bits adicionales para las representaciones intermedias.

Bits adicionales: > precisión

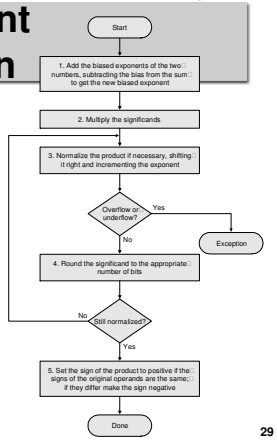
- ***Guard bits:*** son bits a la derecha del bit menos significativo empleados para la normalización (pueden ser importantes en este punto) y son fundamentales en el redondeo.
- En IEEE-754 se tienen tres bits adicionales, llamados ***guard***, ***round***, y ***sticky***. Suelen indicarse como G, R y S.

Anomalías en PF

- Suponga *huge*/*tiny* > 2^{53}
  - tiny + (huge – huge) = tiny + 0 = tiny
  - (tiny + huge) – huge = huge – huge = 0
- La suma no es Asociativa!!!!
  - IEEE 754 requiere que sea conmutativa.
- A veces, el reordenamiento de código efectuado por los compiladores provoca diferentes resultados.
  - Depende del lenguaje o de la "configuración" del compilador.
  - Pequeños cambios pueden tener un efecto gigante.
    - e.g., por ejemplo una excepción de división por cero.

Floating Point Multpication

- Algorithm:



Floating Point Puzzles

– Para cada una de las siguientes expresiones C:
- Justifique si es verdadero o falso.

```
int x = …;
float f = …;
double d = …;
```

Asuma que ni `d` ni `f` es NaN

- `x == (int)(float) x`
- `x == (int)(double) x`
- `f == (float)(double) f`
- `d == (float) d`
- `f == -(-f);`
- `2/3 == 2/3.0`
- `d < 0.0 ⇒ ((d*2) < 0.0)`
- `d > f ⇒ -f < -d`
- `d * d >= 0.0`
- `(d+f)-d == f`

Punto flotante en C

- C garantiza dos niveles:
  - `float` simple precisión.
  - `double` doble precisión.
- Conversiones:
  - *Casting* entre `int`, `float`, y `double` cambia los valores numéricos.
  - `Double` o `float` a `int`.
    - Trunca la parte fraccionaria.
    - Igual que redondeo hacia cero.
    - No definido cuando ocurre fuera de rango.
      - Generalmente *satura* a TMin o TMax.
  - `int` a `double`.
    - Conversión exacta, siempre que `int` tenga un tamaño de palabra ≤ 53 bits.
  - `int` a `float`.
    - Se redondeará según el "modo de redondeo"

Answers to FP Puzzles

• `x == (int)(float) x`	**No: mantisa de 24 bits**
• `x == (int)(double) x`	**Si: mantisa de 53 bits**
• `f == (float)(double) f`	**Si: aumenta precisión**
• `d == (float) d`	**No: pierde precisión**
• `f == -(-f);`	**Si: sólo cambia bit signo**
• `2/3 == 2/3.0`	**No: 2/3 == 1**
• `d < 0.0 ⇒ ((d*2) < 0.0)`	**Si!**
• `d > f ⇒ -f < -d`	**Si!**
• `d * d >= 0.0`	**Si!**
• `(d+f)-d == f`	**No: No asociativa**

Depende del compilador ,2/3 == 0 en **gcc**